

Editor's handbook

For store managers and the staff who keep the site right: fuel prices, pages, promos, holiday hours and emergency notices. No code, no jargon. Every job here is a numbered procedure that ends with how you check it worked.

Version 17 Sep 2026 Audience store managers and CMS editors Editor logins 2 on the current plan
Companion Asset specs & brand guide



01 / how this site stores content

There is no database. Content is files.

Everything you type — a price, a page, a promo, a phone number — is saved as a text file in the project's code repository. That one fact explains most of how this site behaves, including why nothing you do can be permanently lost.

Three doors onto the same act

All three write the same files. There is no separate "CMS content" that can drift out of step with what the developers see.

DOOR	WHO USES IT	BEST FOR
1. TinaCMS the visual editor at /admin	Staff. Sign in with your email — no GitHub account needed.	The daily work: prices, hours, page copy, promos, the emergency notice.
2. An AI session a chat window	Anyone with access to the project's chat sessions.	Bulk changes, anything that is not a field in the CMS, "do all three locations at once".
3. A developer directly in the code	Your engineer.	Design, layout, new kinds of page, anything structural.

Sections 02 and 03 cover door one. Section 10 covers door two. Section 12 is a one-page summary of which door to use for what.

Why files instead of a database matters to you

- **Every change is a recorded commit.** Who changed what, when, and a one-click undo. A wrong fuel price is fixed by reverting to the previous one, not by remembering what it was.
- **Nothing to back up.** The repository is the backup, and it is copied in more than one place already.
- **Deleted is not gone.** Deleting a document removes it from the site, but the document is still in the history and can be brought back. That makes the delete button a safe button rather than a frightening one.
- **The site does not go down if the CMS does.** Published pages are served as static files. If TinaCMS is unreachable you cannot edit, but visitors are unaffected.

How many people can log in

The current plan is TinaCMS Free: two editor logins LOCKED. Growing past two is a purchase, not a settings change.

PLAN	COST	EDITOR LOGINS
Free — what the site runs on today	\$0	2
Team	\$24/mo	3
Team Plus	\$41/mo	5

Prices verified against tina.io, Aug 2026. Team Plus is also the plan that adds an approval step, where one person's edit waits for another person to publish it. The site does not have that today.

Do not share a login

Two seats is tight, and sharing one is the obvious workaround. It removes the thing the commit history is for — knowing who changed the price. If a third person needs to edit, that is a \$24/mo decision, not a password to pass around.

Your first edit, and what “published” means

TinaCMS is a visual editor. You open the real page, click the words you want to change, and type. There is no separate preview to imagine — the page in front of you updates as you type.

Signing in

1. Go to `lummiabay.com/admin`.
2. Enter **your work email address** and follow the sign-in. You do not need a GitHub account and you will never be asked for one.
3. You land in the editor. The left sidebar lists everything you can change, grouped by type.

Making a change on a page

1. Navigate to the page you want, the same way a customer would.
2. **Click the text** you want to change. The sidebar opens on the matching field.
3. Type in the sidebar. The page beside it updates live, so you can see the result before anyone else does.
4. Press **Save**.

How you know it worked

Open the live page in a different browser tab and reload it. If your change is there, it is published. If it is not, check the timing table below before assuming something is broken — most content waits for a rebuild.

How long “published” takes — it depends what you changed

This is the part people get wrong. Three things on this site publish instantly by design; everything else waits about a minute or two for the site to rebuild itself.

WHAT YOU CHANGED	LIVE IN	WHY
A fuel price	Immediately	Prices are read fresh every time a page loads, not baked into the page. There is no rebuild to wait for.
The emergency notice (site alert)	Under a second	Saving it refreshes the site's caches worldwide in about 300 milliseconds, with no rebuild.
A promo starting or ending An hours override starting or ending	On the dot	Whether they are live is worked out per visitor, against today's Pacific date. Nothing has to run at midnight, so nothing can fail to run.
Everything else — page copy, hours, addresses, promo artwork and wording, settings, the watermark	~1-2 min	Your save becomes a commit, the commit triggers a rebuild, the rebuild publishes.

One lag applies to everything, including prices: a page already open on somebody's phone keeps showing the old version until they refresh it. Nothing can change that, and it is the only delay worth explaining to a colleague.

If a rebuild fails, the old site stays up

Visitors see the last good version, not an error page. That is deliberate. But it also means a failure is quiet — if a change you made has not appeared after a few minutes, say so rather than waiting longer. See section 11.

Eight numbers, one form, live the moment you save

This is the thing staff do most, and it is the thing the whole site was built around. The form is designed to be used one-handed on a phone, standing at the pumps, because that is when the price actually changes.

What exists: eight prices, in one place

PLACE	GRADES PRICED ON THE SITE
Exit 260	Regular, Diesel
Minimart	Regular, Diesel
Fisherman's Cove	Regular, Diesel
Truck Stop	Diesel, DEF

Midgrade, premium and ethanol-free are **not** priced on this site. Ethanol-free is still listed as something Fisherman's Cove sells — it just carries no posted price.

Changing a price

1. Go to `/admin` and sign in with your email.
2. In the sidebar, under **Site**, open **Fuel Prices**. The form opens straight away — there is no list to pick from, because there is only one price document.
3. Decide about the checkbox labelled **"Apply one price to all three locations"**.
 - **Checked** — type Regular and Diesel once. All three Locations get that number when you save.
 - **Unchecked** — the three Locations appear separately. Edit whichever ones you need.
4. **Truck Stop is always its own section and is always typed by hand.** Diesel and DEF. The checkbox never touches it.
5. Type dollars with two decimals — `4.05` , not `4.5` and not `$4.05` .
6. Press **Save**. The **updated** date sets itself, for the places you actually changed.

How you know it worked

Open any page of the live site and refresh it hard (hold Shift and click reload). The new number should be there — on *any* page, because the price block is on every page, not only on the fuel prices page.

If the old number is still showing after a hard refresh, that is a bug to report, not a delay to wait out. There is no rebuild happening in the background. Say so straight away.

What the checkbox is, and what it is not

IMPORTANT The checkbox is a typing shortcut, not a price

There is no "all locations" price stored anywhere. Every place stores its own number for every grade. With the box checked, the value you type is written into all three Locations as you save — and from that moment they are three separate numbers that happen to match.

The useful consequence: **unchecking the box can never resurrect an old price.** There is no second, stale value hiding behind the shared one, because no such value is ever stored.

NEVER The Truck Stop is never covered by the checkbox

Truck-lane diesel is not car-lane diesel, and truck prices move on their own schedule. If you changed the Truck Stop you typed it yourself; if you did not type it, it did not change.

The "updated" stamp, and why it matters more than it looks

Each of the four places carries its own **updated** date, and it shows to customers in the expanded price panel. Because publishing is instant, that stamp is now the *only* thing on the page telling a guest how fresh a number is. A missed save leaves no other trace.

When you edit through the CMS the stamp sets itself. **When a price is changed any other way — an AI session, a developer editing the file — the stamp has to be set by hand for every place that changed.** If you ask for a price change in a chat session, that is the one thing worth checking in the result.

If the site cannot reach its price source

The page falls back to the last price it was built with, and shows that price's own updated date, rather than showing a blank space or an error. A guest never sees a hole where a number should be — but they may briefly see an old number with an old date on it. Another reason the stamp is not decoration.

The website's own timing is the only part of a price change this handbook describes. The order your team changes the pumps, the sign and the site in is an operations decision and belongs to you, not to the website.

A page is a document you fill in, not code

You can add a page without an engineer. Pick the right collection in the sidebar, press its create button, fill in the fields, save. What you cannot do — and do not need to do — is lay the page out.

The five collections, and which one you want

COLLECTION	USE IT FOR	WEB ADDRESS
pages	Ordinary pages. About, Rewards, Fuel Prices, Privacy Policy, Contact, and anything new.	/your-slug
infoPages	The page a promo sends people to — the product or offer detail. Header image, title, body, optional hours table, a call to action.	/info/your-slug
promos	The graphic banners themselves. Section 06.	(no page of its own)
tenants	The independent businesses on a Lummi Bay property. Each gets a card on the "Also at Exit 260" page.	(set per tenant)
mainPages	A full replacement home page. Rare. See the warning below.	/ (when switched live)

Locations are not in this list on purpose. There are three Locations and one Truck Stop; the CMS deliberately has no button to create a fourth, because a fourth store is a business event, not a content task.

Adding an ordinary page

1. In the sidebar, open **pages** and press the create button.
2. Give it a **title** and a **slug**. The slug is the bit that shows in the web address, lower case with hyphens — `propane-refills` , not `Propane Refills` .
3. Build the body from the available blocks: rich text, an image banner (its alt text is required), an hours table, the derived location list, a row of buttons, or an FAQ.
4. Save.
5. **Give the page a way in.** This is the step people miss — see section 05.

How you know it worked

After a minute or two, visit the address directly — `lummibay.com/your-slug` . The page should load with the header, the price block, the waterline and the footer already on it. Then follow the link you added in step 5 and make sure it lands there.

What a page can never lose

The header, the waterline, the fuel price block, the back-to-top button and the footer come from the site's layout, not from your page. No page document can move them, restyle them or drop them. You cannot break the furniture by filling in a form, which is why the form is safe to give you.

Two things that are never done

NEVER Never rename or delete `/privacy`

That address is typed into the Rewards app's App Store and Google Play listings. Changing the slug breaks a live app listing; taking the page down can get the app pulled from a store. Treat it more like a phone number than like a page.

Revising the *wording* is fine and expected — the policy is a document that has to stay current. Publish the revision at the same address. Legal text is revised by whoever owns it, not by an engineer and not by an AI session.

NEVER Never delete an info page when its promo ends

When a promo stops running, its info page takes care of itself: nothing on the site links to it any more, it drops out of the sitemap, search engines are told to forget it, and it says plainly that the offer has ended.

It keeps its address. That matters because the address is in Facebook posts, printed QR codes, bookmarks and search results — deleting it turns all of those into a broken site rather than a finished offer. It is also how the offer gets switched back on later.

Deleting an info page outright is allowed when something is genuinely finished forever. That is a deliberate decision you make, not something an expiry date does behind your back.

CAREFUL `mainPages` is a redesign escape hatch

More than one home page may exist; a single setting points at the one that is live, so a replacement can be built in full, reviewed at its own address, and switched over by changing one field — with the old one still there to switch back to.

It is **never scheduled** and it is not how you run a seasonal campaign. A campaign is a promo plus a hero variant (section 06). Reach for `mainPages` only when the page genuinely needs a different layout, and expect a developer to be involved.

The honest chapter: the nav is three items and stays three

"Manage navigation" usually means adding items to the top menu. Here it does not, and pretending otherwise would waste your time. The top nav is a settled decision, and what you manage instead is how people reach everything else.

What is fixed

LOCKED Home • Locations • Truck Stop

Three items, and they are not a CMS field. There is no "add to menu" checkbox on a page, and **a new page never appears in the nav on its own**. Three items fit a phone without a hamburger menu; four were tight once the header also had to carry the logo and the "Get the App" pill.

Fuel Prices used to be the fourth and was deliberately removed. The reason is that the fuel price block is already on every page, so the nav item pointed at a page whose entire contents were already in front of the guest.

The `/fuel-prices` page still exists and is still worth having — it is the page that matches searches like "diesel prices Bellingham". It is reached from **the footer** and from the **"All prices"** panel on the price block, not from the nav.

What you actually control

SURFACE	WHO CHANGES IT	HOW
The three nav items	Nobody, without a decision	Not a field. See below.
The Careers link destination	You	A field in settings. The text stays "Careers".
Social links (Facebook, Instagram, Yelp)	You	A list in settings: platform and web address. An empty list renders no row at all.
Promos	You	Section 06. This is the main way to point people at a new page.
Links inside page copy	You	Rich text, or a row of buttons.
Footer columns, the header, the price block	A developer	Layout, not content.

The Careers link has one rule, and it is about the words

The destination is editable and points off this site, to wherever hiring is actually run. The **link text stays exactly "Careers"** — not "Careers at ...", no company name, no logo, no parenthetical, no tooltip naming where it goes. The moment the label names another company, a signpost becomes an advertisement for that company, which is what the site's standing rule forbids.

There is deliberately no `/careers` page on this site.

So how does a new page get found?

Pick one of these deliberately when you create the page, and do it the same day. A page with no way in is worse than no page.

1. **A promo.** The strongest route, and the one the site is built for: a graphic banner on the pages you choose, pointing at an info page. Section 06.
2. **A footer link.** How `/about` , `/contact` , `/fuel-prices` , `/rewards` and `/privacy` are reached. Adding a new one is a developer change, not a field.
3. **A link from a related page.** The "Also at Exit 260" tenants index is reached from the Exit 260 Location page. A page about propane belongs linked from the Location that sells it.
4. **A link in body copy or a button row.** The cheapest route, and fine for a page that only some people need.

If the nav genuinely must change

It can. It is just not a field, and it should not be. Changing it means re-checking things that were measured against three items — whether the header still fits a 320px phone, and whether the header's motif band still clears the navigation. Raise it with whoever owns the site's architecture; the change gets recorded as a decision, and then a developer builds it. Expect a conversation, not a five-minute edit.

Graphic banners that put themselves up and take themselves down

A promo is a document with artwork, three short lines of text, a date window, and a note of which pages it belongs on. Type the end date when you create it and you never have to remember to take it down.

Creating a promo

1. Create the **info page** first, in `infoPages` — the page the promo sends people to. A promo pointing at nothing does not render.
2. In the sidebar, open **promos** and press create.
3. Fill in the three text fields: **eyebrow** (the small label), **headline** (the offer itself, 28 characters — there is a counter), and **cta** (the button label; it defaults to "See details").
4. Upload the **image** and write its **alt text**. The alt text describes *the offer*, not the picture, because the whole image is a link and the alt text is what a screen reader announces.
5. Set **link** to the info page from step 1.
6. Set **startsAt** and **endsAt**. Both are optional: no start means "running already", no end means "until someone turns it off".
7. Set **placement** — `home`, `all-interior`, or specific pages.
8. Leave **active** on. Set **priority** if several promos are competing for room.
9. Save.

How you know it worked

Open one of the pages you set in **placement**. If the window has started, the promo is there. If it has not, the promos list in the CMS shows the promo's state — **Live**, **Scheduled**, **Ended** or **Off** — so you can confirm it is queued rather than broken.

How expiry works, and why you can trust it

Nothing is scheduled to run at your end date. Instead, every time a visitor loads the page, the site compares today's Pacific date against the window. A promo that ends at the close of Friday stops appearing on Saturday, exactly, with no job to fail and nobody at a keyboard.

Dates are in Pacific time on purpose. "Friday" means Friday here.

Rows: how the region is shaped

The promo region on a page is a list of **rows**, and each row picks how many promos it holds — one, two, three or four across. You build a shape rather than setting a width on each promo and hoping they tile.

- **A row re-divides when one of its promos expires.** A three-across row holding two live promos becomes two halves; holding one, it becomes full width. A hole cannot appear.
- **Five across is not offered.** Twelve does not divide by five, so a five-up row could not re-divide. It is arithmetic, not taste.
- **Rows are capped at six per page.** A row with nothing live renders nothing at all — no gap, no empty box.

- **Extra promos wait their turn.** Anything beyond the available slots stays live in the data and appears as earlier promos expire, in priority order.
- **On a phone every row is one column**, whatever it does on a desktop.

Artwork: the rules that will bite you

NEVER Never bake the offer into the image

Text inside a picture cannot be edited by you, cannot be read by a screen reader, cannot be selected, translated or resized, and does not respond when a customer has enlarged their type. The eyebrow, headline and button are real text laid over the art for exactly that reason. Give the designer the picture; type the words in the form.

NEVER Never name an image file `ad` , `banner` or `sponsored`

Those exact words are on every ad-blocker filter list. A perfectly ordinary in-house promo named `summer-banner.jpg` is invisible to a large share of your customers, and nothing in the site will tell you it happened. Name the file after the offer — `summer-car-wash.jpg` .

CROP LADDER One image has to survive five shapes

Because a row re-divides when a promo expires, a promo supplied for a quarter-width slot can end up displayed full width, with nobody notified. So one master image is supplied and every slot crops from it.

- **Supply one file at 2400 × 1350 (16:9).**
- **Anything that must be seen lives inside the centre 2400 × 480 strip.** That strip is what the full-width slot shows; everything above and below it is breathing room the narrower slots use.
- Compose for the **quarter**, the narrowest slot — the text block takes about 55% of it and leaves the bottom half of the frame for the subject.
- You do not supply a separate phone crop. The phone slot is 16:9, so the master already is the phone crop.

Turning a promo off, and back on

WHAT YOU WANT	WHAT YOU DO
Take it down right now	Set active to false. False beats any date. This is the panic button.
End it earlier than planned	Change endsAt to today.
Run it again later	Clear or extend endsAt and set active back to true. Its info page comes back with it.
See the old ones	The promos list in the CMS is the archive. Every promo is there with its state.
Remove it forever	Delete it. Remember the info page is a separate decision — section 04.

Where a promo lands is decided by the page, not by you

You choose *which* pages a promo appears on. *Where* on the page it sits is set per page type — below the title on most interior pages, below the disclosure on a tenant page, after the phone numbers on Contact. That is not a field, and it is not an oversight: a promo directly under a tenant's name would read as that tenant's offer.

Two related things are set the same way and are worth knowing about. **Hero variants** — a headline and sub-headline on a date window — are how the home page changes for a campaign, using the same date engine as promos. And `/contact` deliberately carries **no promos at all**. [OPEN](#) Whether the home page carries a promo region in the finished build has not been settled — confirm before planning a home-page campaign.

Not a CMS field — but here is how to judge one

The band of small motifs in the header is built in code, not stored as content. You cannot upload a new one or move it from the sidebar. What you *can* do is ask for a change and tell whether what comes back is right, which is what this section is for.

HONEST ANSWER There is no field for this

The motif band is a decorative layer in the site's code. To change it — different motifs, different size, a different colour — **ask in an AI session or ask your developer**. There is no sidebar entry and adding one is not planned.

Section 10 covers how to phrase the request. The rest of this page is so you can check the result without taking anyone's word for it.

The four rules the band must obey

RULE	WHAT IT PREVENTS
20px clear of the right edge of the text navigation	Art crowding the nav, and art sitting behind the nav.
One row of motifs, ever	A size change quietly producing a second row nobody asked for.
Right-aligned	Empty space falling between the nav and the first motif.
Coloured through a mask, with a no-mask fallback	An off-brand colour sneaking in, and — in older browsers — a header full of solid coloured rectangles.

The 20px figure is a **floor, not a target**. The logo's own clear space also applies, so the art starts at whichever of the two lands further right. On a phone there is no text navigation to measure from, so only the logo's clear space binds.

Why the art is never behind the nav

It was built that way once and measured. A lighter band behind the navigation raises the background brightness and eats the contrast of the pale nav text sitting on it. The binding case is the **active** nav item, which already sits on a light highlight: it starts at 5.06:1 against a 4.5:1 minimum, and **fails at 6% band opacity**. At that opacity the art is barely visible — so the entire contrast budget buys something nobody can see.

If someone proposes putting motifs behind the navigation, that is the number to quote back. The only live route to reconsidering it is ownership releasing the reserved deep navy for decoration, which today is reserved for fuel prices and emphasis.

The motifs you see now are placeholders, deliberately obvious ones

They are flat silhouettes — orca, salmon, eagle, canoe — and they look like placeholders on purpose. Coast Salish formline is specific to Lummi Nation, and a convincing imitation is the failure mode, not the goal. Every instance is marked `TODO: replace with approved Lummi art` .

Final art must be authentic, commissioned or tribe-approved before launch. The header band is on every page, so this is the placeholder most likely to survive to launch if nobody chases it. Chase it.

The band is decorative in the technical sense too: it is hidden from screen readers, ignores clicks entirely, and disappears completely for a visitor whose device is set to high contrast. None of that is optional and none of it is yours to change.

The sasquatch — and this one you can turn off yourself

The faint figure at the edge of the page is a content setting, not a piece of code. Switching it off is one checkbox, and it takes effect in under a second. That was the whole point of building it this way: a decoration that needs an engineer to remove is a decoration nobody ever removes.

The fields, in settings

FIELD	WHAT IT DOES
enabled	The off switch. One checkbox. This is the field most people will ever touch.
image	The artwork. Uploading a replacement swaps it — no deploy, no engineer. Read the warning below first.
opacity	How strong the wash is. Not capped — it is the owner's call. See the 10% line below.
side	Left or right. Left is the default.
height	How tall he is, as a percentage of the screen.
crop	How much of the figure runs off the page edge. Set to 0 when the art is already cropped, which the supplied narrow file is.
noBackdrop (on a page, not in settings)	A per-page opt-out, for a page that needs a clean background — a dense table, a map.

Turning him off

1. In the sidebar, open **Settings**.
2. Find the backdrop group and uncheck **enabled**.
3. Save.

How you know it worked

Reload any desktop page. He is gone, within a second — this one does not wait for a rebuild. Do not test it on a phone: he is hidden on phones anyway, so a phone tells you nothing.

The 10% opacity line

The backdrop is a wash sitting underneath text, so it costs contrast on everything on the page. The binding case is the dim label colour `#6B6455` on the page ground `#FBF9F4`, measured against the *darkest* pixel of the artwork, because text can land anywhere over it.

OPACITY	CONTRAST	VERDICT
0%	5.58	Comfortable
8%	4.77	Passes
10%	4.58	The line — still passes, with little left over
12%	4.40	Fails the 4.5:1 minimum
14%	4.21	Fails

The number is not clamped. You can set it past 10% and the field will let you. The form shows the measured contrast live at whatever value you pick and marks where it stops passing, so the choice is made with the cost visible rather than blind. It is the owner's decision, not the software's.

Three things about him that are not negotiable

- **He is hidden on phones.** At phone width the page is wall-to-wall cards and there is no background for him to show on — but the wash would still cost contrast on the text you *can* see.
- **He disappears for high-contrast users.** A visitor who has asked their device for more contrast does not get a wash behind their text. He is also hidden from screen readers and ignores clicks.
- **He is never recoloured.** No tint, no filter. The art renders exactly as supplied, and opacity is the only thing applied to it.

CAREFUL Swapping the art has two consequences

The contrast table above becomes invalid. The 10% line moved from 12% once already, because new art arrived with darker pixels in it than the previous art had. Any change to the **image** field means the line has to be measured again. Ask for that measurement; do not assume 10% still holds.

The raw artwork is never what gets served. The supplied file is 503 × 2948 and 1.85 MB, which is indefensible for a decorative wash. The site converts it at build time and delivers about 128 KB instead — a 93% saving. If you upload replacement art, confirm with a developer that it is still going through that conversion. A raw image file served straight to visitors is a bug, not a preference.

Two tools that get confused with each other

A snowstorm and New Year's Eve are not the same problem, and they do not use the same tool. Using the alert bar for a holiday leaves a wrong opening time sitting on four pages underneath it.

Which one do I want?

	EMERGENCY NOTICE	HOURS OVERRIDE
Use it when	Something is happening now that nobody planned — snow, a power cut, a road closure.	Hours are different on a known date — a holiday, a season.
Scope	Sitewide. Every page, every Location.	One Location (or the Truck Stop) at a time.
Timing	Now. Live in under a second.	Scheduled, typed weeks ahead.
Turning it off	Somebody has to remember.	Automatic, on the end date.
Where it shows	A bar in the sticky header.	The hours line itself, everywhere hours appear.

The alert says *something is happening*. The override makes the hours *correct*. For a same-day hours change, both are worth using together: the override fixes the line, the alert says so at the top of the page.

Publishing an emergency notice

1. In the sidebar, open the **Site Alert** settings.
2. Fill in **headline**, **detail**, and a **link** if there is somewhere useful to send people.
3. Write it leading with what is *true and useful* before the caveat — "All three stores are open" first, then the exception. Name the Location and the road.
4. Set **active** to true.
5. Save.

How you know it worked

Reload any page on the live site. The bar is there, in the header, within a second. There is no rebuild running — if it is not there after a hard refresh, it did not publish.

Taking it down matters as much as putting it up

Set **active** to false. A storm notice still up on a sunny Tuesday costs more credibility than never having posted one. The notice carries its own **updated** stamp, which makes staleness visible — but nothing takes it down for you.

Two other things worth knowing before you need this at 5am. **There is no dismiss button** for visitors, on purpose: the point is that everybody sees it. And the notice is **never used for an offer** — the moment it becomes a marketing surface it stops being believed, and then it does not work for weather either.

Test it on a phone, once, before you need it. The person likely to publish this is likely to be doing it in the dark, from a phone browser, with two editor logins between them and whoever else is asleep. Find out now, not then.

Scheduling different hours for a date

1. Open the Location in the sidebar — or the **Truck Stop** record on Exit 260, which keeps its own hours and its own overrides.
2. Add an entry to **hoursOverrides**.
3. **hours** — the replacement line, written the short way: `6am-6pm` or `Closed` . Keep it the same length as the normal line; there is a character counter, because this line has to fit on a Location card that truncates rather than wraps.
4. **reason** — two or three words, `New Year's Eve` . It shows on the detail page, never on the card.
5. **startsAt** — the first day it applies. Both dates default to today.
6. **endsAt** — the last day it applies. **Required**.
7. Save.

How you know it worked

If the window includes today, check the Location page after the rebuild: it shows **"Today: 6am–6pm — New Year's Eve"** with **"Regular hours 6am–9pm"** underneath. Both lines, because a guest who sees only the short hours has no way to know they are unusual.

If the window is in the future, set both dates to today, look at the page, confirm the wording fits the card, then set the real dates. That is the cheap way to test a holiday in November.

REQUIRED `endsAt` **is not optional, and there is no on/off toggle**

The whole value of this field is that the hours *go back on their own*. An override with no end date is a permanently wrong opening time waiting to happen, and it fails silently, because a wrong time still looks exactly like a time.

There is deliberately no toggle, because a toggle is a thing somebody has to come back and switch off. Instead the dates are editable at any moment, which gives you the same control:

- **Live today only** — leave both dates at today. Type the line, save.
- **Live now, for a while** — leave **startsAt** at today, set **endsAt** to the last day.
- **Start it early** — set both dates in the future and forget it. This is the point.
- **End it sooner** — change **endsAt** to today.
- **Back to normal, right now** — set **endsAt** to yesterday, or delete the override. The normal hours line returns on the next page load.

"Until further notice" has to be written as a date. Pick one far enough out and shorten it later — the override is editable the whole time it is live. A date can be wrong; it cannot be forgotten.

NEVER **Never use an override for a permanent change**

If the store's hours have genuinely changed, edit the **hours** field itself. The override is for the temporary case only, and reaching for it to make a permanent change is the one way to misuse this field — because it will quietly revert on its end date.

Two other rules, so nothing surprises you. If two overrides overlap, **the shorter window wins** — a one-day holiday inside a two-week seasonal change behaves the way whoever typed them meant. And the Truck Stop's hours are separate: the store closing early on a holiday while the diesel lanes stay open all night is the normal case, not an edge case.

The third door: ask in chat

Because content is files in the repository, an AI session can make the same edits you make in TinaCMS — and a few it cannot. It is a third door beside the CMS and your developer, not a replacement for either.

How to phrase a request so it lands

Vague requests come back wrong, and then you check them and ask again. Three habits fix almost everything.

1. **Name the place.** "The Cove", not "the one by the water". "The Truck Stop", not "the truck diesel".
2. **Name the field and the exact new value.** "Set diesel to 4.19", not "diesel is up a bit".
3. **Paste, do not describe.** If you have a price sheet, an address or a block of policy text, paste it in exactly as it is. Describing what it says is where the errors come from.

Worked examples

YOU WANT	SAY SOMETHING LIKE
All three locations, one price	"Set regular to 3.85 and diesel to 4.29 at all three Locations. Leave the Truck Stop alone. Set today's updated stamp on the three you changed."
Truck Stop only	"Truck Stop diesel 4.55, DEF 3.29. Only the Truck Stop. Set its updated stamp to today."
A promo, off now	"Set the 'Summer car wash' promo to active: false. Do not delete it and do not touch its info page."
A holiday, typed early	"Add an hours override to all three Locations: hours 6am–6pm, reason New Year's Eve, startsAt 2026-12-31, endsAt 2026-12-31. Truck Stop stays 24 hours."
A page edit	"On the About page, replace the second paragraph with this text: [paste]. Leave everything else as it is."
A settings change	"Turn the page backdrop off — set enabled to false in the settings backdrop group."
Something that is not a field	"Change the header motif band to use just the canoe, and confirm it still clears the navigation by 20px and is still one row, right-aligned, and masked."

Notice the last one. Naming the rules you expect to be kept is how you get a result you can check — and section 07 is where those rules are written down.

What this door can do that TinaCMS cannot

- **Touch things that are not fields** — the header motifs, the site's colours and spacing, the way a page is laid out.
- **Add a whole new kind of content**, which means a new collection in the CMS, not just a new document.
- **Make many edits at once** — the same holiday override on three Locations, or a text correction across several pages.
- **Explain what happened.** "Why did this promo stop showing?" is a reasonable question to ask in chat and an impossible one to ask a form.

What it cannot do

ALWAYS It cannot approve its own work

An AI session can make a change and tell you it made the change. It cannot be the one who confirms the change is correct, and neither can the person who asked for it without looking.

Somebody opens the live site and checks. Every time. That is not distrust; it is the same rule that applies to a developer and to you.

Two specific things to check, because they are the ones that go wrong quietly:

- **The updated stamp on a price.** A change made outside the CMS does not set it for you. A correct price with a stale date is the one kind of staleness a customer can actually see.
- **That only what you asked for changed.** Look at the neighbouring numbers and the rest of the page, not only the thing you asked about.

It also cannot make a business decision. "Should we drop the price?" is not a question for this door. And anything with a legal owner — the privacy policy text above all — is revised by its owner, not by an AI session and not by an engineer.

Timing through this door

A chat session's edit is an ordinary commit, so the timing table in section 02 applies unchanged: most content waits a minute or two for a rebuild.

Fuel prices should be the exception through this door too, and here is the reasoning — a save in the CMS is itself a commit to the same branch, and the price block does not read the file that was shipped with the site, it asks the content service for whatever is committed now. Neither door is special. What reaches the page is the commit, not who made it.

CONFIRM **One step in that chain has not been measured, and it is the step that would make the difference.** The content service has to notice a new commit and index it before it can serve it. A save made inside the CMS goes through that service, so it is plausibly indexed as part of the save; a commit arriving from outside has to be picked up. That gap is seconds if it exists at all — but "seconds" is a guess until someone changes one real price in chat, watches the live site, and writes down what actually happened. Do that once, early. Until it is written down, look at the live site before you rely on a price you changed in chat.

Triage, before you call anyone

Most of what looks broken is one of six things. Work down this table first; it takes two minutes and it makes the conversation with your developer much shorter.

SYMPTOM	LIKELY CAUSE	WHAT TO DO
The price didn't change	An open tab showing a cached page. Or the save did not go through. Or the checkbox was not doing what you thought.	Hard refresh (Shift + reload) on a page you did not have open. Still wrong? Reopen the Fuel Prices form and read the numbers back. If the form is right and the site is wrong after a hard refresh, report it as a bug — prices do not wait for a rebuild, so there is nothing to wait for.
The promo is still showing	endsAt is later than you think, or the dates are right and it is a stale tab.	Check the promo's state in the CMS list — Live, Scheduled, Ended or Off. To remove it immediately, set active to false; false beats any date.
A promo vanished that shouldn't have	Its info page was deleted. A promo pointing at a missing page is skipped rather than shipping a dead link.	Restore the info page, or repoint the promo's link at a page that exists.
The page is blank or missing	The document was saved without a slug, or the rebuild has not finished, or the rebuild failed.	Wait two minutes and reload. Then check the slug on the document. If the rest of the site is fine but your page is not, the build probably failed — that needs a developer, and visitors are still seeing the last good version meanwhile.
The backdrop is too strong	Opacity is set above the 10% line, or new artwork was uploaded with darker pixels than the last one.	Lower opacity in settings, or uncheck enabled — either is live in under a second. If the art changed, ask for the contrast line to be re-measured (section 08).
The alert won't go away	active is still true, or you are looking at a tab that has not been reloaded.	Set active to false and save, then reload. It clears within a second. If it does not, report it — nothing is queued.
I deleted something	You deleted a document.	It is not gone. Every change is a recorded commit, so the document is still in the history and can be restored. Say what you deleted and roughly when. Do not try to retype it from memory first — restoring it is more accurate.
Two of us edited the same thing	Two editors in the same field at once.	The second person re-does their edit. There is no merge dialog and with two seats this should be rare — but check the field rather than assuming yours won.

The safety net, stated plainly

Every change any of the three doors makes is a recorded commit: who, what, when, and a one-click undo. Nothing you do in the CMS can permanently destroy content, and nothing you do can take the site down — a failed publish leaves the last good version serving.

So the right instinct when something looks wrong is to **say so early**, not to try to quietly fix it. Reverting a known change is a minute's work; untangling three attempted fixes is not.

One page to keep by the till

Who does what

JOB	TINACMS	AI SESSION	DEVELOPER
Fuel prices	Yes — the normal way	Yes	Rarely
Hours, addresses, phone numbers	Yes	Yes	Rarely
Page copy, new pages	Yes	Yes	—
Promos and info pages	Yes	Yes	—
Emergency notice	Yes	Yes	—
Side watermark settings	Yes	Yes	—
Careers link, social links	Yes	Yes	—
Header motifs	No	Yes	Yes
A new footer link	No	Yes	Yes
A new kind of content (a new collection)	No	Yes	Yes
Layout, colours, spacing	No	Yes	Yes
A fourth nav item	No	No	A decision first, then yes

How fast it goes live

CONTENT	LIVE IN
Fuel price	Immediately
Emergency notice on or off	Under a second
Side watermark on, off, or dimmed	Under a second
A promo or hours override starting or ending on its date	On the dot
Everything else	~1-2 min
A page already open on someone's phone	When they refresh

The never-rules, in one place

NEVER

- **Never rename or delete** `/privacy` . It is in two app store listings.
- **Never delete an info page** when its promo ends. It hides itself and keeps its address.
- **Never name an image file** `ad` , `banner` or `sponsored` . Ad blockers eat it silently.
- **Never bake the offer wording into a promo image.** Type it in the fields.
- **Never recolour the logo**, and never recolour the sasquatch.
- **Never add a nav item.** Three, locked. A new page gets a promo or a link instead.
- **Never use an hours override** for a permanent hours change.
- **Never use the emergency notice** for an offer.
- **Never share an editor login.**
- **Never let a placeholder ship.** Anything marked `TOD0: replace with approved Lummi art must be real, approved art before launch.`

ALWAYS

- **Check the live site** after any change made outside TinaCMS.
- **Set the** `updated` **stamp** when a price is changed outside the CMS.
- **Type an** `endsAt` **on every hours override.** It is required for a reason.
- **Give a new page a way in** the same day you create it.
- **Write promo alt text about the offer**, not about the picture.
- **Take the emergency notice down** when it stops being true.
- **Say so early** when something looks wrong. Everything is revertible.

Two facts the footer states, which the site has to keep true

Every page's footer says "**No cookies. Visits counted anonymously.**" That is a checkable claim, not a slogan — the day anything on the site sets a cookie, it becomes a false statement in the footer of every page.

The practical rule for an editor: **anything you are asked to embed is a decision, not a detail.** A social feed, a follow button, a video, a review widget, a chat bubble, a booking button — each of those loads another company's code before a visitor clicks anything. Links out are ordinary content and always fine. Embeds are not yours to add; pass the request on. The one existing exception is the map on Contact, which loads only after a visitor clicks a button that tells them what it will do.

Every procedure in this handbook traces to the project's recorded architecture decisions, its source-of-truth project file, or the fuel-price procedure skill. Where something genuinely has not been settled it is marked `OPEN` or `CONFIRM` rather than guessed at — treat those as questions to answer, not as instructions. If this document and a screen in the CMS disagree, the screen is what staff will follow, so report the difference rather than working around it.